```
import asyncio
import json
import time
import urllib.request
import urllib.parse
import os
import re
import threading
import collections
from http.server import HTTPServer,
BaseHTTPRequestHandler
from socketserver import ThreadingMixIn

# ================= CONFIGURATION
=================
MASTER_SOLANA_WALLET =
"3PURyRLcckKJm4NYKoChomR1qLQDg1M49NmC37QZti
w8"
SOLANA_RPC = "https://api.mainnet-
beta.solana.com"

# FIX: no hardcoded fallback key. Fail
loudly instead of shipping a real secret in
source.
GROQ_API_KEY =
os.environ.get("GROQ_API_KEY", "")
if not GROQ_API_KEY:
    print("⚠️ WARNING: GROQ_API_KEY
environment variable is not set. AI
endpoints will return an error until it is
```

```
configured.")

ROUTER_PORT = int(os.environ.get("PORT",
8080))
SWARM_WORKER_COUNT = 30

DIGITALOCEAN_API_TOKEN =
os.environ.get("DIGITALOCEAN_API_TOKEN",
"")
MAX_SWARM_LIMIT = 100000
CURRENT_ACTIVE_NODES = 1

MONTHLY_SERVER_RESERVE_SOL = 0.035
CLONE_COST_SOL = 0.035
TOTAL_REVENUE_ACCUMULATED = 0.0

task_queue = asyncio.Queue()
MAIN_LOOP = None

PUBLIC_DOMAIN =
os.environ.get("RAILWAY_PUBLIC_DOMAIN",
"solana-swarm-agent-
production.up.railway.app")
PROCESSED_TXS_FILE =
os.environ.get("PROCESSED_TXS_FILE",
"processed_txs.json")

# ================= TELEGRAM APPROVAL
SYSTEM CONFIG =================
TELEGRAM_BOT_TOKEN =
os.environ.get("TELEGRAM_BOT_TOKEN", "")
TELEGRAM_CHAT_ID =
os.environ.get("TELEGRAM_CHAT_ID", "")
SUGGESTION_INTERVAL_SECONDS =
```

```
int(os.environ.get("SUGGESTION_INTERVAL_SEC
ONDS", 21600))  # every 6 hours
PENDING_SUGGESTIONS_FILE =
os.environ.get("PENDING_SUGGESTIONS_FILE",
"pending_suggestions.json")
_telegram_last_update_id = 0

# ================= TWITTER AUTO-POST
CONFIG (only fires after your Telegram YES)
=================
TWITTER_API_KEY =
os.environ.get("TWITTER_API_KEY", "")
TWITTER_API_SECRET =
os.environ.get("TWITTER_API_SECRET", "")
TWITTER_ACCESS_TOKEN =
os.environ.get("TWITTER_ACCESS_TOKEN", "")
TWITTER_ACCESS_SECRET =
os.environ.get("TWITTER_ACCESS_SECRET", "")

# ================= GITHUB AUTO-PR CONFIG
(for new_endpoint_idea, only fires after
your Telegram YES) =================
GITHUB_TOKEN =
os.environ.get("GITHUB_TOKEN", "")
GITHUB_REPO = os.environ.get("GITHUB_REPO",
"")  # format: "username/repo-name"

# FIX: coroutine-result timeouts so a stuck
event loop / hung worker can't wedge the
HTTP thread forever
COROUTINE_RESULT_TIMEOUT_SECONDS = 20

def load_processed_txs() -> set:
    try:
```

```
        with open(PROCESSED_TXS_FILE, "r")
as f:
            return set(json.load(f))
    except (FileNotFoundError,
json.JSONDecodeError):
        return set()

def save_processed_txs(txs: set):
    try:
        with open(PROCESSED_TXS_FILE, "w")
as f:
            json.dump(list(txs), f)
    except Exception as e:
        print(f"⚠️ Could not persist
processed TXs: {e}")

PROCESSED_TXS = load_processed_txs()
PROCESSED_TXS_LOCK = threading.Lock()

RATE_LIMIT_WINDOW_SECONDS = 60
RATE_LIMIT_MAX_REQUESTS = 50
_request_log = {}
_request_log_lock = threading.Lock()
_last_rate_limit_cleanup = time.time()
RATE_LIMIT_CLEANUP_INTERVAL_SECONDS = 300

def is_rate_limited(ip: str) -> bool:
    """FIX: bounded memory. Old/empty IP
entries are purged periodically instead of
    growing _request_log forever
(previously a slow unbounded memory
leak)."""
    global _last_rate_limit_cleanup
    now = time.time()
```

```
    with _request_log_lock:
        timestamps = _request_log.get(ip,
[])
        timestamps = [t for t in timestamps
if now - t < RATE_LIMIT_WINDOW_SECONDS]
        timestamps.append(now)
        _request_log[ip] = timestamps
        limited = len(timestamps) >
RATE_LIMIT_MAX_REQUESTS

        if now - _last_rate_limit_cleanup >
RATE_LIMIT_CLEANUP_INTERVAL_SECONDS:
            for key in
list(_request_log.keys()):
                pruned = [t for t in
_request_log[key] if now - t <
RATE_LIMIT_WINDOW_SECONDS]
                if pruned:
                    _request_log[key] =
pruned
                else:
                    del _request_log[key]
            _last_rate_limit_cleanup = now

    return limited

# ================= NEW: DEMAND TRACKING &
SMART PRICING =================
# FEATURE 1 (smart pricing) + FEATURE 3
(demand tracking): both driven off the
# same request-timestamp log, so we don't
do extra bookkeeping in two places.

_request_timestamps =
```

```
collections.deque(maxlen=5000)   # global
recent request times
_request_timestamps_lock = threading.Lock()

_endpoint_usage_counter =
collections.Counter()          # lifetime
call count per endpoint
_endpoint_usage_lock = threading.Lock()

BASE_PRICE_MULTIPLIER_OVERRIDE = 1.0  #
only ever changed via an approved
pricing_change suggestion

def record_request(service_type: str):
    """Call this once per incoming (paid)
request. Feeds both pricing and stats."""
    now = time.time()
    with _request_timestamps_lock:
        _request_timestamps.append(now)
    with _endpoint_usage_lock:

_endpoint_usage_counter[service_type] += 1

def calculate_smart_surge() -> float:
    """FEATURE 1: Demand-based dynamic
pricing multiplier.
    Looks at how many requests happened in
the last 5 minutes (across all
    endpoints) and scales price up when the
service is in high demand.
    This replaces the old queue-size-only
surge check with something that
    reacts to sustained traffic, not just
an instantaneous queue depth."""
```

```
    now = time.time()
    with _request_timestamps_lock:
        recent = [t for t in
_request_timestamps if now - t < 300]  #
last 5 min
    recent_count = len(recent)

    if recent_count > 50:
        base_surge = 2.0
    elif recent_count > 20:
        base_surge = 1.5
    elif recent_count > 5:
        base_surge = 1.2
    else:
        base_surge = 1.0
    return round(base_surge *
BASE_PRICE_MULTIPLIER_OVERRIDE, 4)

def get_usage_stats() -> dict:
    """FEATURE 3: Snapshot of which
endpoints are most in demand."""
    with _endpoint_usage_lock:
        top =
_endpoint_usage_counter.most_common(10)
        total =
sum(_endpoint_usage_counter.values())
    return {"total_paid_requests": total,
"top_endpoints": top}

# ================ BOUNDED SUGGESTION +
TELEGRAM APPROVAL SYSTEM ================
# This system only ever DRAFTS ideas and
asks a human (you, via Telegram) to
# approve or reject them. Nothing here
```

```
posts to social media, contacts anyone,
# or spends money by itself. The set of
actions it can even suggest is a fixed
# list — it cannot propose or execute
anything outside that list.

ALLOWED_SUGGESTION_TYPES = [
    "pricing_change",        # suggest a
new base price multiplier for one endpoint
    "marketing_draft",       # a
promotional text draft (same as feature 2,
just routed through approval too)
    "new_endpoint_idea",     # a text
description of a possible new paid service
to add
    "community_suggestion",  # names a real
community/platform + a draft message to
post there (you post it manually)
    "usage_insight",         # an
observation about usage patterns,
informational only, auto-"approved" since
it changes nothing
]

def _load_pending_suggestions() -> list:
    try:
        with open(PENDING_SUGGESTIONS_FILE,
"r") as f:
            return json.load(f)
    except (FileNotFoundError,
json.JSONDecodeError):
        return []

def _save_pending_suggestions(suggestions:
```

```
list):
    suggestions = suggestions[-100:]  #
bounded file size
    try:
        with open(PENDING_SUGGESTIONS_FILE,
"w") as f:
            json.dump(suggestions, f,
indent=2)
    except Exception as e:
        print(f"⚠️ Could not save
suggestions: {e}")

def send_telegram_message(text: str) ->
bool:
    """Sends a plain notification/question
to your Telegram chat. Does not
    take any action on your behalf — it
only informs you."""
    if not TELEGRAM_BOT_TOKEN or not
TELEGRAM_CHAT_ID:
        print("⚠️ Telegram not configured
(TELEGRAM_BOT_TOKEN / TELEGRAM_CHAT_ID
missing). Suggestion was only saved to
file.")
        return False
    try:
        url =
f"https://api.telegram.org/bot{TELEGRAM_BOT
_TOKEN}/sendMessage"
        payload = json.dumps({"chat_id":
TELEGRAM_CHAT_ID, "text": text}).encode()
        req = urllib.request.Request(url,
data=payload, headers={"Content-Type":
"application/json"})
```

```
        urllib.request.urlopen(req,
timeout=10)
        return True
    except Exception as e:
        print(f"⚠️ Telegram send failed:
{e}")
        return False

def get_telegram_replies() -> list:
    """Polls Telegram for new messages you
sent the bot (e.g. 'YES 3' or 'NO 3').
    Read-only against Telegram — does not
send anything by itself."""
    global _telegram_last_update_id
    if not TELEGRAM_BOT_TOKEN:
        return []
    try:
        url =
f"https://api.telegram.org/bot{TELEGRAM_BOT
_TOKEN}/getUpdates?offset=
{_telegram_last_update_id + 1}&timeout=5"
        req = urllib.request.Request(url)
        with urllib.request.urlopen(req,
timeout=10) as resp:
            data =
json.loads(resp.read().decode())
        replies = []
        for update in data.get("result",
[]):
            _telegram_last_update_id =
max(_telegram_last_update_id,
update["update_id"])
            msg = update.get("message", {})
            text = msg.get("text", "")
```

```
            chat_id = str(msg.get("chat",
{}).get("id", ""))
            if text and chat_id ==
str(TELEGRAM_CHAT_ID):

replies.append(text.strip())
        return replies
    except Exception as e:
        print(f"⚠️ Telegram poll failed:
{e}")
        return []

def post_tweet(text: str) -> dict:
    """Posts to Twitter/X using OAuth1
user-context credentials. Only called
    from apply_approved_suggestion, i.e.
only after you reply YES on Telegram."""
    if not all([TWITTER_API_KEY,
TWITTER_API_SECRET, TWITTER_ACCESS_TOKEN,
TWITTER_ACCESS_SECRET]):
        return {"status": "ERROR",
"reason": "Twitter credentials not
configured
(TWITTER_API_KEY/SECRET/ACCESS_TOKEN/ACCESS
_SECRET)"}
    try:
        import tweepy
        client = tweepy.Client(
            consumer_key=TWITTER_API_KEY,

consumer_secret=TWITTER_API_SECRET,

access_token=TWITTER_ACCESS_TOKEN,
```

```
access_token_secret=TWITTER_ACCESS_SECRET
        )
        resp =
client.create_tweet(text=text[:280])
        return {"status": "SUCCESS",
"tweet_id": resp.data.get("id")}
    except ImportError:
        return {"status": "ERROR",
"reason": "tweepy not installed. Add
'tweepy' to requirements.txt"}
    except Exception as e:
        return {"status": "ERROR",
"reason": str(e)}

def
create_github_pr_for_endpoint(idea_details:
str, generated_code: str) -> dict:
    """Creates a new branch + commit + pull
request on GitHub containing
    AI-generated code for a new endpoint
idea. Deliberately stops at a PR
    rather than merging to the live branch
directly: this is a real-money
    payment server, and merging is the one
checkpoint left for you to
    glance at before AI-written code
touches production. Only called after
    your Telegram YES."""
    if not GITHUB_TOKEN or not GITHUB_REPO:
        return {"status": "ERROR",
"reason": "GitHub not configured
(GITHUB_TOKEN / GITHUB_REPO missing)"}
    try:
        api_base =
```

```
f"https://api.github.com/repos/{GITHUB_REPO
}"
        headers = {"Authorization":
f"Bearer {GITHUB_TOKEN}", "Accept":
"application/vnd.github+json"}

        def _get(url):
            req =
urllib.request.Request(url,
headers=headers)
            with
urllib.request.urlopen(req, timeout=10) as
resp:
                return
json.loads(resp.read().decode())

        def _post(url, body):
            req =
urllib.request.Request(url,
data=json.dumps(body).encode(),
headers=headers, method="POST")
            with
urllib.request.urlopen(req, timeout=10) as
resp:
                return
json.loads(resp.read().decode())

        def _put(url, body):
            req =
urllib.request.Request(url,
data=json.dumps(body).encode(),
headers=headers, method="PUT")
            with
urllib.request.urlopen(req, timeout=10) as
```

```
resp:
                return
json.loads(resp.read().decode())

        repo_info = _get(api_base)
        default_branch =
repo_info["default_branch"]
        base_ref = _get(f"
{api_base}/git/ref/heads/{default_branch}")
        base_sha = base_ref["object"]
["sha"]

        branch_name = f"ai-suggestion-
{int(time.time())}"
        _post(f"{api_base}/git/refs",
{"ref": f"refs/heads/{branch_name}", "sha":
base_sha})

        file_path =
f"generated_endpoints/{branch_name}.py"
        content_b64 =
json.dumps(generated_code)  # placeholder,
real b64 below
        import base64
        content_b64 =
base64.b64encode(generated_code.encode()).d
ecode()
        _put(f"
{api_base}/contents/{file_path}", {
            "message": f"AI-suggested
endpoint: {idea_details[:60]}",
            "content": content_b64,
            "branch": branch_name
        })
```

```
        pr = _post(f"{api_base}/pulls", {
            "title": f"AI suggestion:
{idea_details[:60]}",
            "head": branch_name,
            "base": default_branch,
            "body": f"Auto-generated from
an approved suggestion.\n\nIdea:
{idea_details}\n\n⚠️ Please review before
merging — this server handles real
payments."
        })
        return {"status": "SUCCESS",
"pr_url": pr.get("html_url")}
    except Exception as e:
        return {"status": "ERROR",
"reason": str(e)}

def apply_approved_suggestion(suggestion:
dict):
    """Executes the effect for the approved
suggestion type. pricing_change
    and marketing_draft take full effect
automatically. new_endpoint_idea
    goes as far as opening a GitHub PR —
merging it is left to you, since
    it's AI-written code touching a live
payment server. usage_insight and
    community_suggestion have no automatic
effect by nature (there's nothing
    safe to automate about picking where
you personally show up)."""
    global BASE_PRICE_MULTIPLIER_OVERRIDE
    stype = suggestion.get("type")
```

```
    if stype == "pricing_change":
        try:
            new_multiplier =
float(suggestion.get("proposed_multiplier",
1.0))
            new_multiplier = max(0.5,
min(new_multiplier, 3.0))  # hard safety
clamp
            BASE_PRICE_MULTIPLIER_OVERRIDE
= new_multiplier
            print(f"✅ Applied approved
pricing change: base multiplier now
{new_multiplier}")
        except Exception as e:
            print(f"⚠️ Could not apply
pricing change: {e}")

    elif stype == "marketing_draft":
        result =
post_tweet(suggestion.get("details", ""))
        if result["status"] == "SUCCESS":
            send_telegram_message(f"🐦
Posted to Twitter (tweet id
{result['tweet_id']}).")
        else:
            send_telegram_message(f"⚠️
Could not post tweet: {result['reason']}")

    elif stype == "new_endpoint_idea":
        idea = suggestion.get("details",
"")
        send_telegram_message("🔧
Generating endpoint code, will open a
```

```
GitHub PR shortly...")

        async def _generate_and_pr():
            loop = asyncio.get_event_loop()
            code_prompt = (
                f"Write a single self-
contained Python function (for a Python
asyncio + "
                f"http.server based API)
implementing this endpoint idea: {idea}\n"
                f"Return ONLY the Python
code, no explanation, no markdown fences."
            )
            code = await
loop.run_in_executor(None,
execute_groq_ai_task, "ask-ai",
code_prompt)
            result = await
loop.run_in_executor(None,
create_github_pr_for_endpoint, idea, code)
            if result["status"] ==
"SUCCESS":
                send_telegram_message(f"✅
PR opened: {result['pr_url']}\nReview and
merge when ready.")
            else:
                send_telegram_message(f"⚠️
Could not create PR: {result['reason']}")


asyncio.run_coroutine_threadsafe(_generate_
and_pr(), MAIN_LOOP)

    else:
```

```
        # community_suggestion,
usage_insight: informational by nature,
nothing to auto-apply
        print(f"✅ Suggestion '{stype}'
marked approved. No automatic action for
this type.")

async def generate_one_suggestion() ->
dict:
    """Asks the AI to produce exactly ONE
suggestion, of one of the allowed
    types, grounded in real usage stats.
The AI picks the type and content,
    but cannot invent a new type outside
ALLOWED_SUGGESTION_TYPES."""
    loop = asyncio.get_event_loop()
    stats = get_usage_stats()
    prompt = (
        f"You help operate a pay-per-call
Solana crypto API (Groq AI text generation,
"
        f"web scraping, Solana DEX price
data). Current usage stats:
{json.dumps(stats)}. "
        f"Suggest exactly ONE concrete
improvement, choosing a type from this
fixed list only: "
        f"{ALLOWED_SUGGESTION_TYPES}. "
        f"Reply ONLY with JSON in this
exact shape: "
        f'{{"type": "<one of the allowed
types>", "reasoning": "<1-2 sentences>", '
        f'"details": "<the actual
suggestion text>", "proposed_multiplier":
```

```
<number, only if type is pricing_change,
else null>}}'
    )
    raw = await loop.run_in_executor(None,
execute_groq_ai_task, "ask-ai", prompt)
    try:
        cleaned = raw.strip().strip("`")
        if
cleaned.lower().startswith("json"):
            cleaned = cleaned[4:].strip()
        parsed = json.loads(cleaned)
        if parsed.get("type") not in
ALLOWED_SUGGESTION_TYPES:
            return None
        return parsed
    except Exception:
        return None

async def autonomous_suggestion_loop():
    """Periodically generates one bounded
suggestion, saves it, and asks you
    on Telegram to approve or reject it.
Also polls for your replies and
    applies ONLY what you approve. Nothing
here acts without your reply."""
    while True:
        await
asyncio.sleep(SUGGESTION_INTERVAL_SECONDS)
        try:
            suggestion = await
generate_one_suggestion()
            if not suggestion:
                continue
            pending =
```

```
_load_pending_suggestions()
            suggestion_id = len(pending) +
1
            suggestion["id"] =
suggestion_id
            suggestion["status"] =
"pending"
            suggestion["created_at"] =
time.time()
            pending.append(suggestion)

_save_pending_suggestions(pending)

            text = (
                f"🤖 Suggestion #
{suggestion_id} ({suggestion['type']}):\n"
                f"
{suggestion.get('details', '')}\n\n"
                f"Reasoning:
{suggestion.get('reasoning', '')}\n\n"
                f"Reply 'YES
{suggestion_id}' to approve, 'NO
{suggestion_id}' to reject."
            )
            send_telegram_message(text)
        except Exception as e:
            print(f"⚠️ suggestion_loop
error: {e}")

async def telegram_reply_listener_loop():
    """Separate loop that just checks for
your YES/NO replies every 30s and
    applies approvals. Kept separate from
the generation loop so replying
```

```
    doesn't have to wait for the next 6-
hour generation cycle."""
    while True:
        await asyncio.sleep(30)
        try:
            replies =
get_telegram_replies()
            if not replies:
                continue
            pending =
_load_pending_suggestions()
            changed = False
            for reply in replies:
                parts = reply.split()
                if len(parts) != 2:
                    continue
                action, id_str =
parts[0].upper(), parts[1]
                if not id_str.isdigit():
                    continue
                sid = int(id_str)
                for s in pending:
                    if s.get("id") == sid
and s.get("status") == "pending":
                        if action == "YES":
                            s["status"] =
"approved"

apply_approved_suggestion(s)

send_telegram_message(f"✅ Suggestion #
{sid} approved and applied.")
                        elif action ==
"NO":
```

```
                                s["status"] =
"rejected"

send_telegram_message(f"❌ Suggestion #
{sid} rejected.")
                            changed = True
            if changed:

_save_pending_suggestions(pending)
        except Exception as e:
            print(f"⚠️
telegram_reply_listener error: {e}")

# ================= 1. REAL SERVICES
EXECUTION ENGINES =================

def fetch_real_solana_dex_data():
    """Fetches real-time DexScreener Solana
Liquidity Data"""
    try:
        url =
"https://api.dexscreener.com/latest/dex/tok
ens/So11111111111111111111111111111111111
1112"
        req = urllib.request.Request(url,
headers={'User-Agent': 'Mozilla/5.0'})
        with urllib.request.urlopen(req,
timeout=5) as resp:
            data =
json.loads(resp.read().decode('utf-8'))
            pairs = data.get("pairs", [])
[:3]
            results = []
            for p in pairs:
```

```
                results.append({
                    "dex": p.get("dexId"),
                    "pair": f"
{p.get('baseToken',
{}).get('symbol')}/{p.get('quoteToken',
{}).get('symbol')}",
                    "price_usd":
p.get("priceUsd"),
                    "volume_24h":
p.get("volume", {}).get("h24"),
                    "liquidity_usd":
p.get("liquidity", {}).get("usd")
                })
            return {"status": "SUCCESS",
"live_dex_signals": results}
    except Exception as e:
        return {"status": "ERROR",
"reason": f"Real-time DEX Fetch Failed:
{str(e)}"}

def scrape_real_website(target_url):
    """Executes Real Web Scraping and
Returns Clean Text"""
    if not target_url or not
target_url.startswith("http"):
        target_url = "https://solana.com"
    try:
        req =
urllib.request.Request(target_url, headers=
{'User-Agent': 'Mozilla/5.0'})
        with urllib.request.urlopen(req,
timeout=7) as resp:
            html = resp.read().decode('utf-
8', errors='ignore')
```

```
            clean_text =
re.sub(r'<script.*?>.*?</script>', '',
html, flags=re.DOTALL)
            clean_text = re.sub(r'<style.*?
>.*?</style>', '', clean_text,
flags=re.DOTALL)
            clean_text = re.sub(r'<.*?>', '
', clean_text)
            clean_text = '
'.join(clean_text.split())[:1500]
            return {"scraped_url":
target_url, "extracted_clean_text":
clean_text}
    except Exception as e:
        return {"status": "ERROR",
"reason": f"Scraping Failed: {str(e)}"}

def execute_groq_ai_task(prompt_type,
input_text):
    """Executes Real LLM Calls using Groq
API"""
    if not GROQ_API_KEY:
        return "GROQ API Key Missing! Set
the GROQ_API_KEY environment variable."

    system_prompts = {
        "prompt-engineer": "You are an
expert AI Prompt Engineer. Convert the user
input into a highly structured, precise
system prompt.",
        "idea-generator": "You are a Web3 &
AI Startup strategist. Generate an
actionable execution blueprint for the
given concept.",
```

```
        "ai-summarize": "You are a summary
specialist. Summarize the text concisely in
key bullet points.",
        "ask-ai": "You are a high-speed
intelligence assistant. Answer the user
query directly and accurately."
    }

    try:
        url =
"https://api.groq.com/openai/v1/chat/comple
tions"
        payload = json.dumps({
            "model": "llama-3.3-70b-
versatile",
            "messages": [
                {"role": "system",
"content": system_prompts.get(prompt_type,
"You are a helpful assistant.")},
                {"role": "user", "content":
input_text if input_text else "Provide a
Web3 AI Agent Strategy"}
            ],
            "max_tokens": 400
        }).encode('utf-8')

        req = urllib.request.Request(url,
data=payload, headers={
            "Content-Type":
"application/json",
            "Authorization": f"Bearer
{GROQ_API_KEY}"
        })
        with urllib.request.urlopen(req,
```

```
timeout=8) as resp:
            res_data =
json.loads(resp.read().decode('utf-8'))
            return res_data['choices'][0]
['message']['content']
    except Exception as e:
        return f"Groq Execution Engine
Fallback Response for: {input_text} (Error:
{str(e)})"

# ================= 2. WORKER AGENT & TASK
EXECUTOR =================

async def worker_agent(worker_id: int):
    while True:
        task = await task_queue.get()
        req_id, service_type, tx_sig,
client_input, response_future = task

        loop = asyncio.get_event_loop()
        output_data = {}

        try:
            if service_type in ["dex-
signal", "volume-spike", "arbitrage-scan"]:
                output_data = await
loop.run_in_executor(None,
fetch_real_solana_dex_data)
            elif service_type == "web-
scraper":
                url_to_scrape =
client_input if client_input else
"https://solana.com"
                output_data = await
```

```
loop.run_in_executor(None,
scrape_real_website, url_to_scrape)
            elif service_type in ["prompt-
engineer", "idea-generator", "ai-
summarize", "ask-ai"]:
                ai_result = await
loop.run_in_executor(None,
execute_groq_ai_task, service_type,
client_input)
                output_data =
{"ai_response": ai_result}
            elif service_type == "pdf-to-
excel":
                output_data = {
                    "parsed_text": "PDF
Document Parsed via Engine",
                    "extracted_fields":
{"status": "PROCESSED", "rows_detected":
15, "format": "JSON"}
                }
            else:
                output_data = {"result":
"TASK_SUCCESSFUL"}

            result_payload = {
                "status": "SUCCESS",
                "service": service_type,
                "worker_id":
f"Worker_{worker_id}",
                "job_id": req_id,
                "onchain_tx": tx_sig,
                "timestamp": time.time(),
                "output": output_data
            }
```

```
        except Exception as e:
            # FIX: a worker-side exception
used to leave response_future unresolved
forever,
            # hanging the HTTP thread
waiting on it. Now we always resolve the
future.
            result_payload = {
                "status": "ERROR",
                "service": service_type,
                "worker_id":
f"Worker_{worker_id}",
                "job_id": req_id,
                "onchain_tx": tx_sig,
                "timestamp": time.time(),
                "error": str(e)
            }

        if not response_future.done():

MAIN_LOOP.call_soon_threadsafe(response_fut
ure.set_result, result_payload)
        task_queue.task_done()

# ================ 3. AUTONOMOUS & PAYWALL
INFRASTRUCTURE ==================

MARKETING_POSTS_FILE =
os.environ.get("MARKETING_POSTS_FILE",
"marketing_posts.json")
MARKETING_GENERATION_INTERVAL_SECONDS =
int(os.environ.get("MARKETING_INTERVAL_SECO
NDS", 21600))  # default: every 6 hours
```

```
def _load_marketing_posts() -> list:
    try:
        with open(MARKETING_POSTS_FILE,
"r") as f:
            return json.load(f)
    except (FileNotFoundError,
json.JSONDecodeError):
        return []

def _save_marketing_post(post_text: str,
stats_snapshot: dict):
    posts = _load_marketing_posts()
    posts.append({
        "text": post_text,
        "generated_at": time.time(),
        "stats_snapshot": stats_snapshot
    })
    posts = posts[-200:]  # keep file
bounded, no unbounded growth
    try:
        with open(MARKETING_POSTS_FILE,
"w") as f:
            json.dump(posts, f, indent=2)
    except Exception as e:
        print(f"⚠️ Could not save marketing
post: {e}")

async def
autonomous_marketing_content_loop():
    """FEATURE 2: Periodically asks the AI
to draft a promotional post about
    whichever endpoint is currently most
popular (using real usage stats),
    and writes it to a local file for a
```

```
human to review and post manually.
    IMPORTANT: this does NOT auto-post to
any social platform. It only
    drafts text. Posting it anywhere is a
deliberate manual step, by design —
    auto-posting to a real account without
review is a separate, riskier
    feature this script does not
implement."""
    loop = asyncio.get_event_loop()
    while True:
        await
asyncio.sleep(MARKETING_GENERATION_INTERVAL
_SECONDS)
        try:
            stats = get_usage_stats()
            if stats["top_endpoints"]:
                top_service =
stats["top_endpoints"][0][0]
                focus_line = f"Focus
especially on the '{top_service}' service,
which is currently the most-used endpoint."
            else:
                focus_line = "No usage data
yet, so write a general introduction post."

            prompt = (
                "Write a short, engaging
tweet (under 280 characters) promoting a "
                "pay-per-call AI API
service that accepts Solana (SOL) crypto
payments "
                "for AI text generation,
web scraping, and Solana DEX price data. "
```

```
                f"{focus_line} Include a
clear call to action. Return only the tweet
text."
            )
            post_text = await
loop.run_in_executor(None,
execute_groq_ai_task, "ask-ai", prompt)
            _save_marketing_post(post_text,
stats)
            print(f"📝 New marketing draft
generated (see {MARKETING_POSTS_FILE}).
Review before posting anywhere.")
        except Exception as e:
            print(f"⚠️
marketing_content_loop error: {e}")

async def autonomous_directory_indexer():
    print("📡 DISCOVERY ENGINE: Broadcaster
Online for Real Groq-Powered AI
Services...")
    agent_manifest = {
        "name": "Solana Swarm Real AI Agent
(Groq Powered)",
        "symbol": "SWARM-GROQ-AI",
        "version": "6.0.0",
        "wallet": MASTER_SOLANA_WALLET,
        "base_url":
f"https://{PUBLIC_DOMAIN}",
        "protocol": "x402-solana-paywall"
    }
    directory_nodes =
["https://api.virtuals.io/v1/agents/registe
r", "https://solana-ai-
registry.net/v1/index"]
```

```
    while True:
        await asyncio.sleep(120)
        for node in directory_nodes:
            try:
                payload =
json.dumps(agent_manifest).encode('utf-8')
                req =
urllib.request.Request(node, data=payload,
headers={"Content-Type":
"application/json"})
                loop =
asyncio.get_event_loop()
                await
loop.run_in_executor(None, lambda:
urllib.request.urlopen(req, timeout=5))
            except Exception:
                pass

async def autonomous_life_support_loop():
    # NOTE: left the actual scaling
decision/logic untouched — only wrapped in
error handling
    # so one bad request doesn't kill the
whole background loop. Whether to add a
human
    # approval step and a low hard cap on
MAX_SWARM_LIMIT is a decision for you to
make.
    global CURRENT_ACTIVE_NODES,
TOTAL_REVENUE_ACCUMULATED
    while True:
        await asyncio.sleep(60)
        try:
            current_funds =
```

```
TOTAL_REVENUE_ACCUMULATED
            if current_funds <
MONTHLY_SERVER_RESERVE_SOL:
                continue
            surplus_funds = current_funds -
MONTHLY_SERVER_RESERVE_SOL
            if surplus_funds >=
CLONE_COST_SOL and CURRENT_ACTIVE_NODES <
MAX_SWARM_LIMIT:
                if await
spawn_child_vps_node():
                    CURRENT_ACTIVE_NODES +=
1

TOTAL_REVENUE_ACCUMULATED -=
(MONTHLY_SERVER_RESERVE_SOL +
CLONE_COST_SOL)
        except Exception as e:
            print(f"⚠️ life_support_loop
error: {e}")

async def spawn_child_vps_node() -> bool:
    if not DIGITALOCEAN_API_TOKEN:
        return False
    url =
"https://api.digitalocean.com/v2/droplets"
    payload = json.dumps({
        "name": f"swarm-node-
{int(time.time())}", "region": "nyc3",
        "size": "s-1vcpu-1gb", "image":
"ubuntu-22-04-x64"
    }).encode()
    req = urllib.request.Request(url,
data=payload, headers={
```

```
        "Content-Type": "application/json",
"Authorization": f"Bearer
{DIGITALOCEAN_API_TOKEN}"
    })
    try:
        loop = asyncio.get_event_loop()
        return await
loop.run_in_executor(None, lambda:
urllib.request.urlopen(req,
timeout=10).status in (200, 201, 202))
    except Exception:
        return False

async def
verify_onchain_payment(tx_signature: str,
price_sol: float) -> bool:
    global TOTAL_REVENUE_ACCUMULATED
    # FIX: check-then-add on PROCESSED_TXS
was not thread/coroutine-safe against
concurrent
    # requests reusing the same signature;
guarded with a lock.
    with PROCESSED_TXS_LOCK:
        if not tx_signature or tx_signature
in PROCESSED_TXS:
            return False

    payload = json.dumps({
        "jsonrpc": "2.0", "id": 1,
"method": "getTransaction",
        "params": [tx_signature,
{"encoding": "jsonParsed", "commitment":
"finalized",
"maxSupportedTransactionVersion": 0}]
```

```
    }).encode()
    req =
urllib.request.Request(SOLANA_RPC,
data=payload, headers={"Content-Type":
"application/json"})
    try:
        loop = asyncio.get_event_loop()
        result = await
loop.run_in_executor(None, lambda:
json.loads(urllib.request.urlopen(req,
timeout=10).read()))
        tx_data = result.get("result")
        if not tx_data or
tx_data.get("meta", {}).get("err"):
            return False

        account_keys =
tx_data["transaction"]["message"]
["accountKeys"]
        pre_balances = tx_data["meta"]
["preBalances"]
        post_balances = tx_data["meta"]
["postBalances"]
        wallet_index = next((idx for idx,
key in enumerate(account_keys) if
(key.get("pubkey") if isinstance(key, dict)
else key) == MASTER_SOLANA_WALLET), None)

        if wallet_index is None:
            return False

        received_sol =
(post_balances[wallet_index] -
pre_balances[wallet_index]) / 1_000_000_000
```

```
        if received_sol < price_sol:
            return False

        with PROCESSED_TXS_LOCK:
            if tx_signature in
PROCESSED_TXS:
                # Another concurrent
request already claimed this tx while we
were verifying.
                return False
            PROCESSED_TXS.add(tx_signature)

save_processed_txs(PROCESSED_TXS)
        TOTAL_REVENUE_ACCUMULATED +=
received_sol
        return True
    except Exception as e:
        print(f"❌ Payment Error: {e}")
        return False

class ThreadedHTTPServer(ThreadingMixIn,
HTTPServer):
    daemon_threads = True

class
PaywallRouterHandler(BaseHTTPRequestHandler
):
    def log_message(self, format, *args):
        return

    def _send_json(self, code, obj):
        try:
            self.send_response(code)
            self.send_header('Content-
```

```
Type', 'application/json')
            self.end_headers()

self.wfile.write(json.dumps(obj).encode())
        except Exception:
            pass

    def do_GET(self):
        # FIX: entire handler wrapped so an
unexpected exception returns a 500
        # instead of killing the connection
/ leaving the client hanging.
        try:
            self._handle_get()
        except Exception as e:
            print(f"❌ Unhandled request
error: {e}")
            try:
                self._send_json(500,
{"error": "INTERNAL_SERVER_ERROR"})
            except Exception:
                pass

    def _handle_get(self):
        client_ip = self.client_address[0]
        if is_rate_limited(client_ip):
            self.send_response(429)
            self.end_headers()
            return

        # FEATURE 1: demand-based pricing
instead of just instantaneous queue depth
        surge = calculate_smart_surge()
        routes = {
```

```
            "/v1/prompt-engineer":
("prompt-engineer", round(0.002 * surge,
4)),
            "/v1/idea-generator": ("idea-
generator", round(0.002 * surge, 4)),
            "/v1/ask-ai": ("ask-ai",
round(0.001 * surge, 4)),
            "/v1/web-scraper": ("web-
scraper", round(0.002 * surge, 4)),
            "/v1/ai-summarize": ("ai-
summarize", round(0.003 * surge, 4)),
            "/v1/pdf-to-excel": ("pdf-to-
excel", round(0.005 * surge, 4)),
            "/v1/dex-signal": ("dex-
signal", round(0.001 * surge, 4)),
            "/v1/arbitrage-scan":
("arbitrage-scan", round(0.003 * surge,
4)),
            "/v1/volume-spike": ("volume-
spike", round(0.005 * surge, 4))
        }

        if self.path in ["/", ""]:
            self._send_json(200, {
                "status": "ONLINE",
                "version": "6.1.0-SMART-
PRICING-ENGINE",
                "paid_endpoints":
list(routes.keys()),
                "free_endpoints":
["/v1/health", "/v1/stats",
"/v1/marketing"]
            })
            return
```

```
        if self.path == "/v1/health":
            self._send_json(200,
{"revenue_sol":
round(TOTAL_REVENUE_ACCUMULATED, 4),
"tx_count": len(PROCESSED_TXS)})
            return

        if self.path == "/v1/stats":
            # FEATURE 3: which endpoints
are most in demand (free, no payment
needed)
            self._send_json(200,
get_usage_stats())
            return

        if self.path == "/v1/marketing":
            # FEATURE 2: latest AI-drafted
marketing posts, for a human to
review/copy.
            # Free endpoint, read-only,
does not post anywhere by itself.
            posts = _load_marketing_posts()
            self._send_json(200, {"drafts":
posts[-10:]})
            return

        if self.path == "/v1/suggestions":
            # Bounded suggestion system:
view pending/approved/rejected suggestions.
            # Approval itself only happens
via your Telegram YES/NO replies.
            suggestions =
_load_pending_suggestions()
```

```
            self._send_json(200,
{"suggestions": suggestions[-20:]})
            return

        parsed_path =
urllib.parse.urlparse(self.path).path
        if parsed_path in routes:
            service_name, price =
routes[parsed_path]
            tx_signature =
self.headers.get('X-TX-Signature')
            client_input =
self.headers.get('X-Input-Data', '')

            if not tx_signature:
                res = {"status":
"402_PAYMENT_REQUIRED", "service":
service_name, "price": f"{price} SOL",
"pay_to": MASTER_SOLANA_WALLET}
                self._send_json(402, res)
                return

            # FIX: bounded wait on payment
verification instead of an unbounded
.result()
            try:
                fut_verify =
asyncio.run_coroutine_threadsafe(verify_onc
hain_payment(tx_signature, price),
MAIN_LOOP)
                verified =
fut_verify.result(timeout=COROUTINE_RESULT_
TIMEOUT_SECONDS)
            except asyncio.TimeoutError:
```

```
                self._send_json(504,
{"error": "PAYMENT_VERIFICATION_TIMEOUT"})
                return
            except Exception as e:
                self._send_json(500,
{"error": "PAYMENT_VERIFICATION_FAILED",
"detail": str(e)})
                return

            if not verified:
                self._send_json(403,
{"error": "INVALID_OR_USED_TX"})
                return

            # FEATURE 1 + 3: count this as
a real, paid request for pricing/demand
stats
            record_request(service_name)

            response_future =
MAIN_LOOP.create_future()
            req_id = f"job_{int(time.time()
* 1000)}"

asyncio.run_coroutine_threadsafe(task_queue
.put((req_id, service_name, tx_signature,
client_input, response_future)), MAIN_LOOP)

            # FIX: bounded wait on job
execution instead of an unbounded .result()
            try:
                fut_res =
asyncio.run_coroutine_threadsafe(asyncio.wr
ap_future(response_future), MAIN_LOOP)
```

```
                result =
fut_res.result(timeout=COROUTINE_RESULT_TIM
EOUT_SECONDS)
            except asyncio.TimeoutError:
                self._send_json(504,
{"error": "JOB_EXECUTION_TIMEOUT",
"job_id": req_id})
                return
            except Exception as e:
                self._send_json(500,
{"error": "JOB_EXECUTION_FAILED", "detail":
str(e)})
                return

            self._send_json(200, result)
        else:
            self.send_response(404)
            self.end_headers()

def start_server():
    server = ThreadedHTTPServer(('0.0.0.0',
ROUTER_PORT), PaywallRouterHandler)
    server.serve_forever()

async def main():
    global MAIN_LOOP
    MAIN_LOOP = asyncio.get_running_loop()
    for i in range(1, SWARM_WORKER_COUNT +
1):

asyncio.create_task(worker_agent(i))

asyncio.create_task(autonomous_life_support
_loop())
```

```
asyncio.create_task(autonomous_directory_in
dexer())

asyncio.create_task(autonomous_marketing_co
ntent_loop())

asyncio.create_task(autonomous_suggestion_l
oop())

asyncio.create_task(telegram_reply_listener
_loop())
    if TELEGRAM_BOT_TOKEN and
TELEGRAM_CHAT_ID:
        send_telegram_message("🚀 Swarm
agent is online. I'll send you bounded
suggestions periodically — reply YES <id>
or NO <id> to act on them.")
    print(f"🚀 GROQ REAL-AI SWARM AGENT
LIVE | Workers: {SWARM_WORKER_COUNT} |
Port: {ROUTER_PORT}")
    loop = asyncio.get_event_loop()
    await loop.run_in_executor(None,
start_server)

if __name__ == '__main__':
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        pass
```